

Daily Scholar Papers Report — 2026-06-05

2026-06-05

Daily Scholar Papers Report — 2026-06-05

[Download PDF](#)

Window covered: 2026-06-04 → 2026-06-05 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

A thin three-alert window, but the central hit is on-target. The Outstanding paper is **Jaralyzer** (ICSE 2026, Schott et al., Paderborn + SAP + Fraunhofer IEM), the first OSS-dependency scanner that operates directly on Java *bytecode* — closing the long-standing source-vs-bytecode mismatch that has limited Eclipse Steady’s coverage. On the unmodified Achilles benchmark Jaralyzer reports 28 more true positives and 29 fewer false positives than Steady on 56 popular Java OSS components, and is the only tool among six evaluated to handle all four Dann-et-al. modification types (re-compilation / Uber-JAR / bare Uber-JAR / re-packaging) with at most 6% miss on the hardest case, while running >10× faster than Steady. The Keep is a brief surfacing of the 2016 Ellis–Solar-Lezama–Tenenbaum NIPS paper on sampling from posteriors over programs, re-indexed today via Solar-Lezama’s Scholar feed. One off-topic alert (verifiable-database cryptography) was skipped. No user-curated self-emails arrived.

Outstanding: 1 · **Keep:** 1 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	Bytecode-centric Detection of Known-to-be-vulnerable Dependencies in Java Projects (Jaralyzer)	Schott, Ponta, Fischer, Klauke, Bodden	ICSE 2026 / arXiv 2510.19393	PDF
2.1	Sampling for Bayesian Program Learning	Ellis, Solar-Lezama, Tenenbaum	NIPS 2016	PDF

1. Outstanding

1.1 · SCA / SUPPLY CHAIN · First fully bytecode-centric Java SCA — +28 true, −29 false vs Eclipse Steady on 56 unmodified deps, only tool to handle all four dependency-modification types ($\leq 6\%$ miss on re-packaging) at $>10\times$ the speed

1.1 [Bytecode-centric Detection of Known-to-be-vulnerable Dependencies in Java Projects \(Jaralyzer\)](#) — Schott, Ponta, Fischer, Klauke, Bodden (Paderborn + SAP + Fraunhofer IEM), ICSE 2026 / arXiv 2510.19393 (Oct 2025)

Authors: Stefan Schott, Serena Elisa Ponta, Wolfram Fischer, Jonas Klauke, Eric Bodden **Venue:** ICSE 2026 (Research Track); arXiv 2510.19393v1 [cs.SE] **Code:** [stschott/jaralyzer](https://github.com/stschott/jaralyzer)

Problem. 71% of typical Java code comes from OSS dependencies (Endor Labs 2023). Existing dependency scanners fall into two camps that each break on common Java practice. (a) *Metadata-based* scanners (OWASP DC, OSV, Dependabot) compare an SBOM against advisories — they over-report on multi-module projects (Spring’s 20 modules share a single CVE entry) and lag advisory updates (median 25 days; Maven 41-day average). (b) *Code-centric* scanners — currently Eclipse Steady (Ponta et al., ICSME 2018 / EMSE 2020) — compare a fix-commit database against the dependency’s *source code*, which is often unavailable or doesn’t match the bytecode on Maven Central. Worse, Dann et al. (TSE 2021) identified 67,196 Maven Central artifacts containing modified copies of just 254 vulnerable classes across four common modification patterns. Modifications are the norm, not the exception.

Approach (§3, Figure 1). Two stages: (i) build a knowledge base from fix commits, by *compiling only the source files touched by each commit in isolation*, normalising the resulting bytecode with jNorm (ECOOP 2024), generating Code Property Graphs, and extracting Bowman-Huang string triplets; (ii) scan a project’s bytecode dependencies by matching constructs to the KB by Fully Qualified Name, then comparing per-method triplet sets against the KB to decide vulnerable/fixed.

The headline engineering win is the *compile-only-touched-files* heuristic with Jess (ICSME 2024) as a fallback: it raises fix-commit compilation success on Project KB from a 14.9% baseline (Tufano-et-al.-style whole-repo compilation) to **664/733 = 90.6%**. Without this, code-centric scanning of fix commits is simply not viable at scale.

Decision rule (§3.3.3, paper’s notation). Given a CPG g with triplet set $T = \{(n_s, e_l, n_t) \mid \text{edge } e \in g\}$, and pre-fix / post-fix CPGs g_{vul} , g_{fix} :

- $CT = T_{\text{vul}} \cap T_{\text{fix}}$ (context — unchanged by the fix)
- $PT = T_{\text{fix}} \setminus T_{\text{vul}}$ (positive — added by the fix)
- $NT = T_{\text{vul}} \setminus T_{\text{fix}}$ (negative — removed by the fix)

A scanned method’s triplet set T_m is classified vulnerable iff $|NT \cap T_m| \geq |PT \cap T_m|$ (more similar to pre-fix). For add-only fixes ($NT = \emptyset$), require $|PT \cap T_m| \geq \theta_{PT}$. Re-packaging mode unqualifies all triplets, additionally requires $|CT \cap T_m| > \theta_{CT}$, and validates class-context similarity via sibling-method / sibling-field overlap above θ_{CC} . Defaults tuned empirically: $\theta_{PT} = 0.5$, $\theta_{CC} = 0.3$, $\theta_{CT} = 0.3$.

This decomposition is the clean approximation — it side-steps the NP-complete subgraph-isomorphism problem with a constant-time set-overlap check.

Headline numbers (§4.1–4.4).

Setting	Jaralyzer	Eclipse Steady	Other scanners
Unmodified (RQ2), unique TPs / FPs vs Steady on shared 664-CVE KB	+35 TP / 4 FP	+7 TP / 33 FP	—
Net delta vs Steady on unmodified	+28 TP, -29 FP	—	—
Modifications handled (RQ1)	all 4 types, ≤6% miss on type-4	types 1–3 only	OWASP DC type-4 partial (22 misses); Dependabot / OSV / commercial fail on types 2–4
Runtime on 56 deps	131 s (full) / 63 s (default only)	1,387 s	OWASP DC 9 s; OSV 5 s; commercial 30 s
Recall × precision on Dann et al.'s small benchmark (corrected ground truth)	100% / 100% across all modification types	—	—

Where Jaralyzer wins on jackson-databind. 30 of the 35 unique TPs vs Steady are jackson-databind CVEs whose fixes change *field initialisers* or *static initialisers* (regex updates, deserialisation blocklists). The Java compiler inlines this into methods, so bytecode preserves the change — Steady can't see it because it compares bytecode against source.

Threats to validity (§5). Achilles OSS pre-2021 only; θ thresholds tuned on Achilles (overfitting risk); 61 Project KB CVEs uncompileable (excluded from RQ2 — a hybrid metadata-fallback à la Steady could recover them).

Closing line (verbatim, §7): “These results position Jaralyzer to be the code-centric dependency scanner of choice for Java projects, especially when it comes to identifying modified dependencies.”

Why it matters. Three reusable moves beyond the headline. (1) *Compile only the fix-touched files, not the whole repo* lifts compilation success from 14.9% to 90.6% and is reusable by any code-centric SCA / patch-presence tool that needs to materialise historical fix-commit bytecode. (2) *Triplet decomposition into CT / PT / NT with the simple $|NT \cap T_m| \geq |PT \cap T_m|$ rule* is a teachable approximation that replaces the NP-complete subgraph match with constant-time set overlap — and the empirical results show it is competitive. (3) *Re-packaging detection via unqualified triplets + class-context overlap* is the first practical handling of Maven Shade Plugin transformations in an OSS scanner; the same idea transfers to Android TPL re-packaging (cf. PHunter ISSTA 2023). The complete evaluation matrix (5 scanners × 5 benchmarks = 30 reports, with TP/FP adjudicated against four advisories simultaneously) is also a model template for SCA tool evaluation when no single ground truth can be trusted.

2. Keep

2.1 · PROGRAM SYNTHESIS · 2016 NIPS paper resurfacing via MIT DSpace re-index — sample from program posteriors by combining SAT-based synthesis with random parity constraints, with provable approximation guarantees

2.1 [Sampling for Bayesian Program Learning](#) — Ellis, Solar-Lezama, Tenenbaum (MIT BCS + CSAIL), NIPS 2016

Authors: Kevin Ellis, Armando Solar-Lezama, Joshua B. Tenenbaum **Venue:** 30th Conference on Neural Information Processing Systems (NIPS 2016), Barcelona **Source:** Surfaced today via MIT DSpace institutional re-index in the Solar-Lezama Scholar feed — not a new publication.

Summary. Introduces the problem of sampling programs from posteriors $p(x) \propto 2^{-|x|} \cdot 1[\text{spec}(x)]$ conditioned on input/output examples (or other specifications). A sketch defines the finite program set S and the description-length prior $2^{-|x|}$. The algorithm encodes $x \in S$ as assignments to n Boolean decision variables, then combines constraint-based program synthesis (SAT-solver enumeration of $X \subseteq S$) with sampling via random parity constraints (Chakraborty/Meel/Vardi-style hashing) to draw samples provably close to the true posterior. Implemented as **PROGRAMSAMPLE** and evaluated on 22 program-learning problems across text editing (string-edit DSL) and computer-aided programming (recursive list manipulation including counting, sorting, reversing).

Why it lands in Keep, not Outstanding. Followed-researcher signal (Solar-Lezama) but the paper is from 2016; it's appearing today only because Scholar's MIT DSpace crawler re-indexed the institutional copy. Worth keeping the trace because the SAT-encoding + parity-constraint sampling pattern remains the cleanest scaffold for "synthesise *distributions* of programs, not single programs" — a frame that is increasingly relevant to LLM-as-program-synthesiser systems that need calibrated uncertainty over multi-candidate patches (e.g., yesterday's SEC-bench Pro digest showed agents submitting 38 PoC candidates for one verified bug).

Cross-Paper Synthesis

A thin day, but the Outstanding hit is squarely in the digest's running thread on OSS supply-chain analysis. Jaralyzer is the cleanest statement to date that *the Java SCA pipeline has been wrong about the basic data unit*: Eclipse Steady operates on source because fix commits live in source; Jaralyzer flips that and operates on bytecode because deployed dependencies live in bytecode, bridging the gap by compiling only the *fix-touched files* and normalising the result with jNorm. The 90.6% compilation success (vs 14.9% baseline) is the load-bearing engineering win; the CPG-triplet decision rule is the load-bearing scientific simplification.

The Jaralyzer result also echoes the bytecode-level patch-presence line — PPT4J (ICSE 2024), BScout (USENIX 2020), P³ (ICSE 2024), Osprey for binaries, PHunter (ISSTA 2023) for Android — all variants of "tell whether deployed code contains the vulnerable pre-patch version", using progressively more semantics-aware features (line numbers → symbolic signatures → CPG triplets). Jaralyzer's distinctive contribution is to *also* solve the upstream pipeline (materialise fix-commit bytecode reliably), which the patch-presence line has generally side-stepped by assuming the reference bytecode is already available.

The Ellis et al. re-surfacing is a minor footnote, but its frame — *sample from a posterior over programs with provable guarantees* — is a useful counterweight to the current LLM-rejection-sampling default in agentic patch synthesis.

Writing & Rationale Insights

The Jaralyzer paper is worth studying as a writing template even apart from its technical content.

RQ ordering signals the contribution. RQ1 is “How does Jaralyzer compare on *modified* dependencies?” — putting the harder, contribution-defining case first; RQ2 (“unmodified”) is positioned as the “but also” win on the already-solved case. This is the right ordering when an established state-of-the-art tool (Steady) exists on the easier sub-problem.

Refusing the official ground truth. Achilles ships a ground truth, but Schott et al. found two of 15 entries are wrong and explicitly opt out, replacing it with a head-to-head per-tool agreement methodology. This is honest and reproducible; the “unclear” bucket for inter-advisory disagreements is the right escape valve. Worth citing as a template for *evaluating SCA tools when the ground truth itself is suspect*.

Multi-advisory adjudication (GA + NVD + Snyk + Project KB). The “TP iff *all* listing advisories agree it affects this artifact-version” rule is conservative and well-defined.

Closing the research-to-infrastructure loop. They reported 9 ground-truth corrections + 52 newly agreed cases back to the Achilles maintainers (PRs 44 & 45) — converting a research result into community-benchmark improvement.